

Práctica 3: Estimación de Pi con Monte Carlo y pthreads

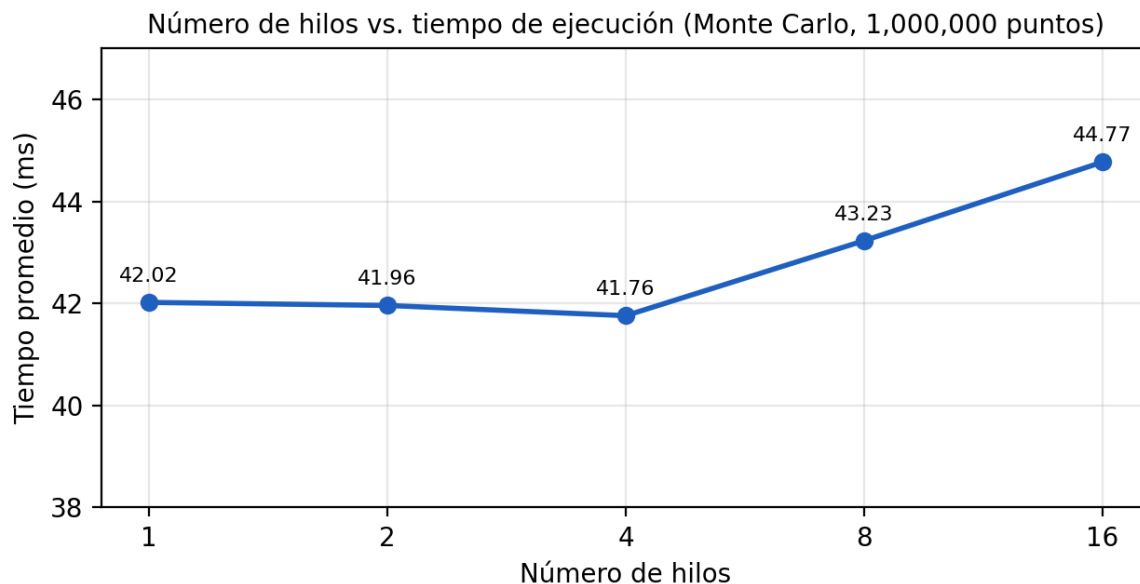
Resultados de rendimiento según el número de hilos

Se compiló **hilo.c** con el número de hilos como argumento y se midió el tiempo de la fase de creación, ejecución y unión de hilos con `clock_gettime`. La lógica del programa no se modificó. Cada configuración se ejecutó 30 veces con 1,000,000 de puntos totales, repartidos en partes iguales entre los hilos.

1. Tabla de resultados

Hilos	Tiempo promedio (ms)	Tiempo mínimo (ms)	Pi promedio
1	42.02	39.19	3.142993
2	41.96	38.22	3.142749
4	41.76	39.51	3.143682
8	43.23	41.84	3.140886
16	44.77	41.36	3.142263

2. Gráfica: número de hilos vs. tiempo de ejecución



3. Conclusión

Con **1, 2 y 4 hilos** el tiempo fue prácticamente el mismo (alrededor de 42 ms); la diferencia de 0.2 a 0.3 ms está dentro del ruido de medición. Nominalmente 4 hilos fue la configuración más rápida (41.76 ms), pero no representa una mejora real. Con **8 y 16 hilos** el tiempo aumentó ligeramente

(43.23 y 44.77 ms) porque crear y planificar más hilos añade costo sin que haya más trabajo útil que repartir.

Por qué no hubo aceleración: las mediciones se hicieron en un entorno con un solo núcleo, por lo que los hilos se turnan en el mismo procesador y no se ejecutan en paralelo real. En un equipo con varios núcleos se espera una mejora al pasar de 1 a 2 y 4 hilos, con el mejor rendimiento cerca del número de núcleos disponibles y un estancamiento o empeoramiento al superarlo.

Nota: todos los hilos comparten el estado de `rand()`, lo que puede causar contención y limitar la escalabilidad. Una alternativa es usar `rand_r` con una semilla distinta por hilo. El valor de Pi se mantuvo cerca de 3.14 en todas las configuraciones.